

MEMO-Appflow: 基于真实 eBPF 证据与端侧大模型的 Android 实时应用预测和系统调度

Jingyi Guo

2026-07-01

摘要

MEMO-Appflow 是面向 Android 的端侧、基于本地大模型推理的应用预测与系统调度软件。它从 app 与操作系统两侧获取信息：在手机内持续采集真实 app 序列、eBPF 内核证据和系统状态，每 3 分钟压缩成 MAPLE 场景输入。大模型据此预测接下来可能使用的 Top-3 应用，并用 bitmask 选择调度动作，形成“从 app + OS 到 app + OS”的闭环：既更新桌面 Widget、优化 app 推荐体验，也执行预热、内存整理和服务刷新，改善 OS 状态。最新 9 分钟真机实验显示，Top-3 会随窗口刷新，scheduler_bits 已触发 warm launch、memory trim 和 service manager 调度。

目录

1	实时任务队列设计	3
2	MEMO-Appflow: 基于真实 eBPF 证据与端侧大模型的 Android 实时应用预测和系统调度	4
2.1	摘要	4
2.2	1. 产品动机	5
2.3	2. 系统目标	5
2.4	3. 总体架构	6
2.5	4. eBPF 与系统证据采集	7
2.6	5. 数据模型与压缩	7
2.7	6. MAPLE 推理与 prompt 裁剪	8
2.8	7. 真实 App 推荐与 Widget	9
2.9	8. ActionExecutor 与调度动作	9
2.10	9. 实验环境	10
2.11	10. 9 分钟实验设计	10
2.12	11. 9 分钟实验结果	11

2.13 12. 调度动作结果 11

2.14 13. 产品效果解读 12

2.15 14. 与压力 A/B 实验的关系 13

2.16 15. 局限性 13

2.17 16. 后续工作 13

2.18 17. 结论 14

1 实时任务队列设计

MEMO-Appflow 的实时模式不是“收完一段数据后停止等待推理”，而是把手机端采集和 MAPLE 推理解耦成一个有界流水线。系统持续运行一个采集流，每 3 分钟生成一个窗口。窗口结束后，当前窗口被压缩为 MAPLE scenario，并提交给唯一的 MAPLE worker；与此同时，下一个 3 分钟窗口立即继续采集。这样用户继续使用手机时，采集不会因为上一轮模型推理而暂停。

为了避免长期后台运行时线程或任务无限堆积，实时模式只保留三类运行单元：一个 collector loop、一个 MAPLE worker、一个 latest pending slot。如果 MAPLE 推理慢于窗口节奏，系统不会给每个窗口新开线程，也不会无限排队历史窗口，而是只保留最新待推理窗口。新窗口仍然以自己的 3 分钟 app/eBPF/system evidence 为主输入；EMA memory 只作为有界历史辅助信息，不替代当前窗口。

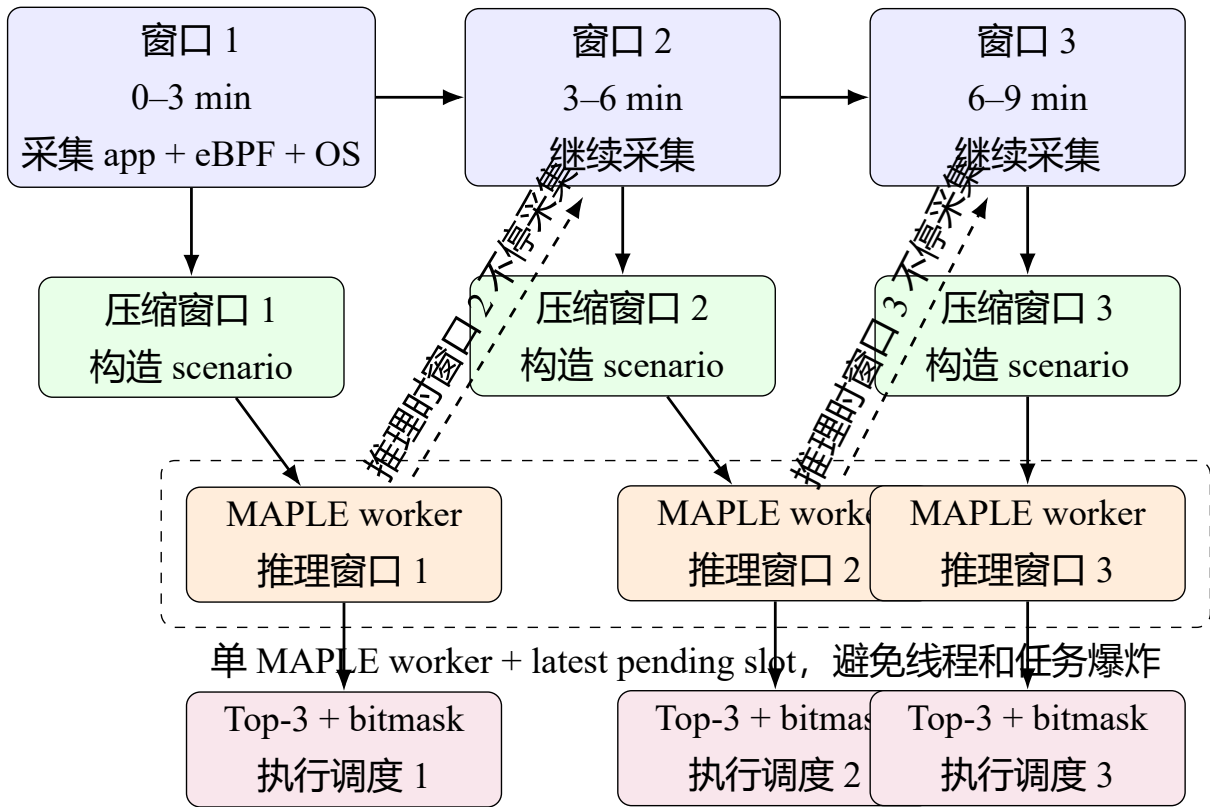


图 1: 3 分钟窗口采集与 MAPLE 异步推理流水线

表 1: 实时任务队列的关键约束

设计点	作用
3 分钟窗口	固定节奏聚合真实 app 序列、eBPF 事件和系统状态, 避免每个事件都触发推理。
单 MAPLE worker	控制端侧推理并发, 避免本地小模型占用过多 CPU 和内存。
latest pending slot	当 MAPLE 慢于采集节奏时只保留最新窗口, 避免历史任务无限排队。
EMA memory	保留有界历史偏好, 但当前窗口仍然是主输入。
ActionExecutor	把 Top-3 与 scheduler_bits 转化为受控系统动作。

2 MEMO-Appflow: 基于真实 eBPF 证据与端侧大模型的 Android 实时应用预测和系统调度

作者: Jingyi Guo

实验日期: 2026-07-01

实验设备: rooted Pixel 5, arm64-v8a

项目仓库: MEMO-Appflow

2.1 摘要

移动设备的资源调度长期处在一个矛盾中：一方面，用户希望应用切换、相机调用、网络访问和媒体播放足够快；另一方面，手机的内存、CPU、I/O、温度和电量都受到严格约束。传统应用预测如果只看用户最近打开了哪些 app，容易把问题简化成 app 序列分类；这种方式可以做出一个 Top-3 推荐列表，但很难解释为什么某个应用应该被预热，也很难判断当前系统是否适合执行预热、内存整理或服务刷新。MEMO-Appflow 的核心目标是把预测从“浅层 app 序列”推进到“真实系统证据驱动的端侧推理与调度”：在 Android 设备内部持续采集前台 app 序列、eBPF 内核级事件和系统状态，按 3 分钟实时窗口压缩成结构化场景，交给本地 MAPLE 大模型推理。MAPLE 输出两类结果：接下来可能使用的 Top-3 真实应用，以及一组 scheduler_bits，用于控制 Android 端预先编程好的调度动作。ActionExecutor 根据 bitmask、当前内存压力、温度和前台状态执行安全动作，例如 Top-1 warm launch、memory trim、网络统计刷新和 service manager refresh。最新 9 分钟真实手机实验表明，MEMO 能在手机端连续采集真实证据，MAPLE 推理期间不停止下一窗口采集，Top-3 推荐能随窗口变化，且模型输出的 scheduler bitmask 已经真实触发系统调度动作。这说明 MEMO-Appflow 已从 host 脚本原型推进为一个端侧可运行的产品闭环。

2.2 1. 产品动机

Android 手机的使用场景有明显的连续性。用户打开聊天软件后，可能马上拍照、发图、查看链接或切换到浏览器；用户看视频时，网络接收、媒体解码和界面刷新会持续活跃；用户进行支付或登录时，Binder 服务调用、网络访问和安全相关服务会比普通浏览更重要。如果系统能够提前理解这些信号，就可以更合理地安排资源：在压力允许时预热即将使用的应用，在显示/UI 繁忙时降低预热强度，在内存紧张时少做激进动作，在网络活跃时优先保留通信或浏览器类 app。

仅仅记录 app 序列是不够的。比如两个用户都打开了 Chrome，背后的系统状态可能完全不同：一个人在看网页文本，另一个人在播放视频；一个设备内存充足，另一个设备已经接近回收压力；一个场景主要是网络接收，另一个场景可能是 Binder 与显示合成频繁。如果预测模型只看到“Chrome -> QQ -> Camera”这种序列，它很难判断下一步应该优化网络、相机、显示还是内存。MEMO-Appflow 因此把 eBPF 作为核心证据来源，直接从内核事件和系统状态中观察更深层的行为。

本项目还强调“端侧产品闭环”。早期原型可以在电脑上通过 adb 和 Python 脚本收集数据、处理 JSON、调用模型，再把结果写回手机。但这样的系统不是一个真正的手机产品：拔掉线就不能用，迁移到真机也不成立，用户无法通过桌面 Widget 获得实时推荐，系统调度也没有真正发生。因此当前版本将采集、解析、压缩、MAPLE 调用、Top-3 映射、调度执行和 Widget 展示全部放到 Android 设备内部。Host 只负责安装 APK、推送模型或查看结果，不承担产品逻辑。

2.3 2. 系统目标

MEMO-Appflow 的目标可以分成五层。

第一，真实采集。系统必须从 Android 设备内部获取真实 app 使用序列、真实 eBPF 证据和真实系统状态，而不是手写 demo 或 host 脚本伪造数据。采集对象包括 memory、battery、network、camera/media、display/UI、process/service 和 Binder 等维度。

第二，结构化压缩。eBPF 原始事件数量很大，很多行会重复出现。如果把所有原始行直接送给模型，会浪费上下文并降低信噪比。因此 MEMO 会按时间窗口、事件类型、进程、证据类别和重复次数压缩数据。重复事件不会全部展开，而是保留代表性样本和 count，让模型知道“同一类事情在某个时间段重复发生了多少次”。

第三，端侧 MAPLE 推理。MAPLE 是唯一预测入口。Android 端把压缩后的 scenario JSON 写入设备本地路径，调用本地 GGUF 模型和 MAPLE C++ demo。模型输入不仅包含 eBPF，也包含前台 app 序列、系统状态、实时窗口摘要和有界 EMA memory。

第四，真实应用推荐。用户不能看到进程名、Binder 服务名或 SurfaceFlinger 这类内部概念。MEMO 使用 PackageManager 扫描设备上真实安装的应用，把 MAPLE 的资源类别或 App id 映射为真实可启动 app，输出 Top-3，展示在 App 和桌面 Widget 中。

第五，系统调度动作。模型输出不能只停留在文字建议。当前 MAPLE 会输出 scheduler_bits，Android 端把它解释为一组预定义动作的开关。ActionExecutor 再根据当前压力和安全规

则执行动作, 例如 warm launch、memory trim、network stats refresh、service manager refresh, 或者在不安全时跳过 drop cache。

2.4 3. 总体架构

当前产品链路如下:



这个链路的关键是“一切都在手机内部发生”。App 可以一直在后台收集数据。每个

3 分钟窗口结束后，当前窗口被压缩并提交给一个 MAPLE worker；与此同时，下一个窗口继续采集。这样推理不会阻塞用户继续使用手机。为了避免线程爆炸，系统只保留一个采集流、一个 MAPLE worker 和一个 latest pending slot。如果模型运行慢于窗口节奏，不会无限积压旧窗口，而是保留最新待推理窗口。新的窗口仍然使用自己的 3 分钟 app/eBPF/system evidence 作为主输入，EMA memory 只是辅助历史上下文。

2.5 4. eBPF 与系统证据采集

MEMO 的证据来源分为 app 序列、eBPF 事件和系统状态三类。

app 序列来自 Android 前台状态采样。系统记录某个时间点前台包名、应用 label 和活动窗口变化，把连续相同 app 合并成 app segment。这样模型能知道用户在某个时间段里真实打开了哪些 app，哪些 app 是连续使用，哪些是短暂切换。

eBPF 事件来自设备侧 collector。当前系统不再依赖 bpfttrace 脚本，而是向 raw eBPF/libbpf/CORE 方向迁移，采集输出仍兼容现有 MEMO_* 事件格式。事件类别包括：

- MEMO_OPENAT：文件或设备节点访问。
- MEMO_SENDTO / MEMO_RECVFROM：网络发送与接收，尤其 UDP 相关行为。
- MEMO_BINDER：Android 服务 IPC 和 Binder 相关活动。
- MEMO_SCHED：调度与进程运行信号。
- MEMO_PROCESS_FORK / MEMO_PROCESS_EXEC / MEMO_PROCESS_EXIT：进程生命周期。

系统状态包括 memory、battery、network、camera/media、display/UI、process/service 等。memory 侧重点是 MemAvailable、swap、reclaim、LMKD 压力；battery 侧重点是电量、充电状态、温度；network 侧重点是 socket、UDP 收发、网络统计；camera/media 关注相机服务、MediaCodec、音视频相关服务；display/UI 关注 SurfaceFlinger、RenderThread 和输入/滑动相关信号；process/service 关注前台 app、关键系统服务和 Binder 调用。

这些数据不会原样全部送入模型。原始 trace 可能在 3 分钟内达到上万行，其中大量重复行来自同一类系统活动。MEMO 会按时间窗口聚合，把重复事件变成“事件类型 + 证据类别 + 进程 + count + 时间范围”。这样既保留“发生了什么”和“发生了多少次”，又避免模型被重复文本淹没。

2.6 5. 数据模型与压缩

MAPLE 的输入不是单一表，而是一个 scenario JSON。它包含以下主要部分。

第一，observed_app_sequence。这里保存真实前台 app 序列，包含时间戳、包名、label、持续时间和切换次数。它回答“用户刚才实际用了哪些 app”。

第二，timeline_windows。这里按时间片对齐 app 序列和 eBPF 事件。比如同一个 3 分钟窗口可以被切成多个小窗口，每个小窗口记录当时主要 app、主要 eBPF 类别、网络收发、Binder 活动、进程变化和显示/UI 信号。这个结构解决了用户提出的关键问题：MAPLE 不应该只拿 eBPF，也应该看到 app 信息，并且 app 和 eBPF 要按时间戳对齐。

第三, `system_evidence`。这里汇总 `memory`、`battery`、`network`、`camera/media`、`display/UI`、`process/service`。它回答“设备当前适不适合预热、是否有压力、应该偏向什么调度策略”。

第四, `realtime_recent_window`。这里描述最近 3 分钟窗口的核心摘要, 包括 `app labels`、`top eBPF signals`、`top categories`、`raw event count`、`app segment count` 和压缩信号数量。

第五, `realtime_memory`。这是有界 EMA memory。它不会无限增长, 而是维护一组带衰减的历史计数。近期证据权重更高, 旧证据逐渐衰减。这样模型能获得历史偏好, 但不会把旧数据堆到 `prompt` 里。

第六, `available_scheduler_actions`。这里列出 Android 端已经实现的动作。MAPLE 不直接执行任意 `shell` 命令, 而是在这些预先编程好的安全动作中输出 0/1 bitmask。

这种数据模型的优势是层次清楚。原始 eBPF 用于真实性和可追溯性; 压缩统计用于模型推理; `app sequence` 用于用户行为理解; `system state` 用于判断动作是否安全; `scheduler action contract` 用于把大模型输出限制在可控范围内。

2.7 6. MAPLE 推理与 prompt 裁剪

端侧小模型的上下文和推理速度都有限, 因此 MEMO 对 MAPLE 输入做了裁剪。裁剪不是随意丢信息, 而是把低密度原始文本压缩成高密度结构。比如 1000 条重复的 `MEMO_RECVFROM` 不会逐条出现, 而是压缩成“Network IO 在某时间窗口出现 N 次, 主要进程是什么, 是否与当前前台 app 同步”。这样会损失单条事件的细节, 但保留了调度所需的语义: 网络是否活跃、发生在哪段时间、和哪个 app 使用阶段有关、强度大概是多少。

当前 MAPLE 主要完成三个任务。

第一, 资源方向判断。模型根据 `app/eBPF/system evidence` 判断接下来更可能需要哪些资源, 例如 Network IO、Camera Service、Media Codec、Android Service IPC、Memory Management 等。

第二, Top-3 行为预测。`prompt` 明确要求模型预测“用户接下来要使用的三个 app”。模型可以输出内部 App id 或类别, Android 端再映射到真实安装应用。这个设计避免把模型绑定到某台设备固定 app 列表。

第三, 调度 bitmask 输出。MAPLE 输出一个长度为 9 的 `scheduler_bits` 字符串, 每一位对应一个预定义动作。比如 100000000 表示执行 bit0 的 `warm_launch_top1`; 101010100 表示执行 bit0、bit2、bit4、bit6。Android 端再根据安全策略决定是否实际执行。例如 bit4 是 `drop cache`, 但如果当前内存不是 `critical`, `ActionExecutor` 会跳过它, 避免无意义或有风险的系统动作。

这种设计让大模型参与调度, 但不让大模型直接拥有任意系统权限。模型负责“选择策略”, Android 端负责“安全执行”。

2.8 7. 真实 App 推荐与 Widget

用户看到的是 Top-3 真实应用，而不是进程名或系统服务。AppIdMapping 使用 PackageManager 扫描设备安装的 launchable apps，根据应用 label、包名、类别、历史使用和 MAPLE 输出进行映射。比如网络类场景可以映射到 Chrome、Messages、QQ；通信类可以映射到 QQ、Messages；camera/media 类可以映射到 Camera 或媒体相关 app。

Widget 是产品化重点。实时模式每 3 分钟刷新一次 Top-3，桌面 Widget 展示应用图标和名称，点击即可打开。App 内部则展示更详细的系统判断、证据摘要和执行动作。eBPF 原始记录不会直接堆在首页，而是通过报告或 debug 入口查看，并用可读格式呈现。

这个设计区分了普通使用者和开发/实验人员。普通使用者只关心“MEMO 推荐了什么、更新时间是什么、有没有优化系统”；开发者可以进入报告看 raw trace、scenario JSON、MAPLE output 和 action logs。

2.9 8. ActionExecutor 与调度动作

ActionExecutor 是把预测变成系统影响的关键层。没有它，MAPLE 只是输出文本；有了它，模型结果才能改变设备状态。

当前预定义 9 个 scheduler actions：

bit	action_id	作用
0	warm_launch_top1	预热 Top-1 推荐 app，然后回 HOME
1	warm_launch_top2_if_idle	空闲时预热 Top-2
2	trim_memory_low_priority_apps	对低优先级候选做内存 trim
3	kill_selected_background_apps	关闭被选中的低优先级后台包
4	drop_cache_if_critical_memory	仅 critical 内存压力时 drop cache
5	refresh_network_stats	刷新网络统计
6	refresh_service_manager	查询/刷新 service manager
7	reduce_prewarm_when_display_busy	显示/UI 繁忙时降低预热强度
8	skip_camera_prewarm_when_too_hot	温度高时跳过 camera/media 预热

每个动作都不是盲目执行。比如 warm launch 会先检查推荐列表是否存在、内存和温度是否允许；drop cache 必须在 critical memory 下才执行；Top-2 预热需要设备空闲；显

示/UI 忙时会降低预热强度，避免造成 jank。ActionExecutor 的职责不是无条件服从大模型，而是在模型策略和系统安全之间做落地。

这也是 MEMO 与普通推荐系统的区别：它既预测 app，也把预测转化为系统层面的动作，并记录动作是否执行、耗时多少、为什么跳过。

2.10 9. 实验环境

最新实验在 rooted Pixel 5 上完成，CPU ABI 为 arm64-v8a。设备已授权 root，MEMO APK 安装在手机上，MAPLE runtime 和 GGUF 模型放在设备本地路径。Host 不处理数据，只执行三类辅助操作：安装 APK、触发 Activity 入口、拉取结果文件。

实验入口为：

```
adb shell am start -n com.memoos/.MainActivity --es memo_action com.memoos.action.REAL
```

这个命令等价于从 App 前台触发实验按钮。它不是直接启动非 exported service，而是让 MainActivity 将 memo_action 转成内部 foreground service。因此实验路径与用户点击产品入口一致。

结果文件保存到：

```
docs/real_device_experiments/realtime_top3_shift_2026_07_01/latest_realtime_top3_shift  
docs/real_device_experiments/realtime_top3_shift_2026_07_01/artifacts/
```

其中 artifacts 包含每个 phase 的 scenario_prompt.json、context.json 和 maple_output.txt。这保证实验透明：我们可以看到模型输入是什么、输出是什么、耗时多少、最后执行了哪些动作。

2.11 10. 9 分钟实验设计

实验包含三个完整的 3 分钟窗口，总长度约 9 分钟。每个窗口自动反复打开手机上真实存在的 app，让最近使用行为出现明显变化，从而验证 Top-3 是否会随窗口更新。

第一阶段是 network/browser window。系统反复打开 Chrome、QQ、Magisk 等应用，目标是产生较强网络、Binder、进程和显示证据。

第二阶段是 camera/photo window。设计目标是偏向相机/图片场景，但当前 adaptive app scanner 没有稳定选到 Google Camera，而是选择了 QQ、Chrome、Tencent Meeting 等真实应用。这个结果暴露了 app scanner 对 camera 类应用选择不够强的问题，但不影响实验真实性，因为实际行为仍然来自真实 app。

第三阶段是 media/communication window。系统反复打开 Chrome、QQ、Tencent Meeting，产生通信、媒体、网络和服务调用信号。

每个窗口结束后，MEMO 立即把窗口压缩成 MAPLE scenario 并提交给后台 MAPLE worker；下一窗口继续采集。因此，MAPLE 推理的 20-40 秒耗时不会阻塞下一窗口的数据收集。

2.12 11. 9 分钟实验结果

实验结果如下：

阶段	raw eBPF	app 段数	MAPLE 耗时	MAPLE		
	事件			App id	Top-3	scheduler_bits
network	19263	52	29106 ms	220	Messages / Camera / Chrome	100000000
camera	19289	55	37196 ms	115	QQ / Mes-sages / Camera	100000000
media_comm	19260	54	23839 ms	220	Messages / Camera / QQ	101010100

Top-3 变化如下：

实验开始前：Settings / Messages / Chrome
窗口 1 后：Messages / Camera / Chrome
窗口 2 后：QQ / Messages / Camera
窗口 3 后：Messages / Camera / QQ

Top-3 在三个窗口之间变化了 2 次。这说明推荐不是硬编码结果，而是随着最近窗口的真实 app sequence、eBPF/system evidence 和 MAPLE 推理更新。

MAPLE 耗时分别是 29.1 秒、37.2 秒、23.8 秒。这个耗时对“用户等待一次按钮结果”来说偏长，但实时模式并不要求用户等待。用户可以继续使用手机，推理在后台完成，完成后 Widget 和动作状态更新。当前设计更接近后台智能调度，而不是前台同步问答。

2.13 12. 调度动作结果

实验最关键的结果是：MAPLE 的 scheduler_bits 已经进入 Android 产品路径，并触发了真实动作。

第一阶段 network：

scheduler_bits=100000000
widget_update: ok, 39 ms
latency_policy: ok, MAPLE 异步完成，耗时 29.1s
maple_scheduler_plan: ok, 9 个预定义动作中选择 1 个执行
maple_warm_launch_top1: ok, 1715 ms, 预热 Messages 并返回 HOME

第二阶段 camera:

```
scheduler_bits=100000000
widget_update: ok, 35 ms
latency_policy: ok, MAPLE 异步完成, 耗时 37.2s
maple_scheduler_plan: ok, 9 个预定义动作中选择 1 个执行
maple_warm_launch_top1: ok, 1312 ms, 预热 QQ 并返回 HOME
```

第三阶段 media_comm:

```
scheduler_bits=101010100
widget_update: ok, 49 ms
latency_policy: ok, MAPLE 异步完成, 耗时 23.8s
maple_scheduler_plan: ok, 9 个预定义动作中选择 4 个执行
maple_warm_launch_top1: ok, 1310 ms, 预热 Messages 并返回 HOME
maple_trim_memory: ok, 414 ms, 对 Camera/QQ 等低优先级候选做 memory trim
maple_drop_cache: skipped, 当前内存不是 critical, 所以安全策略阻止 drop cache
maple_service_manager_refresh: ok, 304 ms, 刷新 service manager
```

这里可以看到两层逻辑。第一，模型确实输出了 bitmask；第二，Android 端不是盲目执行所有动作，而是根据安全条件执行或跳过。第三阶段中 bit4 请求 drop cache，但因为当前内存不是 critical，所以系统跳过。这种记录非常重要，因为它证明 MEMO 不是为了展示而强行执行危险命令，而是在真实设备上做受控调度。

2.14 13. 产品效果解读

从用户角度看，MEMO 做了三件事。

第一，它观察最近 3 分钟使用行为。观察对象包括真实 app 使用、网络收发、Binder 服务调用、进程运行、文件访问、显示/UI 和系统状态。用户不需要理解 eBPF，只需要知道 MEMO 在后台观察“刚才这段使用给手机带来了什么系统压力”。

第二，它更新 Top-3 推荐。实验中 Top-3 从 Settings/Messages/Chrome 变为 Messages/Camera/Chrome，再变为 QQ/Messages/Camera，最后变为 Messages/Camera/QQ。桌面 Widget 可以把这些推荐直接展示出来，帮助用户快速打开可能要用的 app。

第三，它执行调度动作。实验中 MEMO 预热了 Messages、QQ、Messages，并在第三阶段执行 memory trim 和 service manager refresh。这些动作对手机的影响是：让高概率应用更可能保留在热路径上，减少后续冷启动成本；在合适的时候整理低优先级候选，避免资源被不重要对象占用；刷新系统服务信息，让后续判断更准确。

需要注意的是，单次 9 分钟实验不能证明所有场景都提升性能。它证明的是“闭环已经真实成立”：真实采集、端侧推理、Top-3 更新、bitmask 调度和动作记录都跑通。性能改善需要更长时间、更复杂压力条件和更多 A/B 对照实验来评估。

2.15 14. 与压力 A/B 实验的关系

之前的手机压力 A/B 实验关注“开启 MEMO 后，用户使用其他 app 时系统压力是否下降”。这类实验通常比较启动耗时、wait time、CPU busy、iowait、memory drop、reclaim 和 jank rate 等指标。它回答“MEMO 对手机整体使用压力有没有帮助”。

本次 9 分钟实时实验回答另一个问题：“MEMO 的实时产品闭环能不能运转，并且大模型能不能真的驱动调度动作”。两个实验互补。A/B 实验更像性能评估，9 分钟实验更像产品链路验证。一个系统要上线，两者都需要：既要证明能跑通，也要证明在特定压力场景下有收益。

从当前结果看，MEMO 已经具备进一步做大规模压力实验的基础。因为调度动作已经由 MAPLE bitmask 驱动，后续可以比较“只推荐不调度”“推荐 + rule-based 调度”“推荐 + MAPLE bitmask 调度”三种配置对压力指标的影响。

2.16 15. 局限性

当前系统仍有几个明确局限。

第一，adaptive app scanner 还需要改进。第二阶段原本设计为 camera/photo，但自动选择没有稳定命中 Google Camera。这说明应用类别映射和候选排序仍需结合更多 package metadata、历史打开记录 and 用户安装 app 特征。

第二，小模型的 scheduler bits 还不够丰富。前两段都输出 100000000，只触发 Top-1 warm launch。第三段输出 101010100，已经更丰富，但仍然需要更多 prompt 和样本来让模型更准确地区分网络、相机、显示、内存和服务压力。

第三，当前 9 分钟实验是受控极端条件。它适合证明 Top-3 会变化和动作会执行，但不等同于全天自然使用。后续需要更长时间后台运行，记录用户自然行为下的推荐变化、动作频率、温度、电量和 jank。

第四，root 权限仍是部署前提。eBPF 和部分系统调度动作需要 root 或系统级权限。真机产品化需要明确权限策略，或者在可控实验环境、系统 app、定制 ROM 或开发者设备中运行。

第五，MAPLE 推理延迟仍然较高。当前一次推理约 24-37 秒。实时模式通过异步 worker 避免阻塞用户，但如果要更高频刷新，需要继续优化模型大小、prompt 长度和推理后端。

2.17 16. 后续工作

后续优先级包括：

1. 改进 app scanner，让 Camera、Gallery、Payment、Video 等类别更稳定映射到真实 app。
2. 优化 scheduler prompt，使模型输出更细粒度 bitmask，而不是频繁只选 bit0。
3. 增加更长时间自然使用实验，记录 1 小时、半天或全天的 Top-3 更新和动作执行。

4. 做严格 A/B 消融：去掉 app sequence、去掉 eBPF、去掉 memory、去掉 network、去掉 scheduler bits，比较预测和系统压力指标。
5. 量化调度收益：启动 TotalTime、WaitTime、CPU busy、iowait、reclaim、jank、温度、电量消耗和动作执行开销。
6. 继续推进 raw eBPF/libbpf/CO-RE，减少对不稳定工具链的依赖，提高 ARM/Linux 兼容性。
7. 优化 UI，让用户只看到 Top-3、更新时间、系统优化摘要和可读证据入口，避免暴露过多 JSON。

2.18 17. 结论

MEMO-Appflow 当前已经从“能在电脑上跑脚本”的原型，推进到“能在真实 Android 手机内部运行的端侧产品闭环”。系统能够持续采集真实 app 序列和 eBPF 证据，把数据压缩成 MAPLE scenario，在设备本地运行大模型，输出 Top-3 真实应用和 scheduler bitmask，并由 ActionExecutor 执行系统调度动作。最新 9 分钟实验显示，Top-3 推荐随 3 分钟窗口变化，MAPLE 输出的 scheduler_bits 成功进入 Android 路径，并触发了 Top-1 warm launch、memory trim 和 service manager refresh 等动作。

这个结果的意义不在于宣称所有手机场景都已经被优化，而在于证明了产品方向成立：MEMO 可以把深层系统证据、大模型推理和 Android 调度动作连接起来。相比只看 app 序列的 baseline，MEMO 的输入更接近真实系统状态；相比只输出预测文本的模型 demo，MEMO 的输出已经能改变设备行为。后续工作应围绕更严谨的长期实验、更强的 app 映射、更细粒度的调度 bitmask 和更低延迟的端侧推理继续推进。